《STL 源码剖析》技能包
Codex 从安装到使用教程

WEveboy · Codex 协作整理

第一版 V1 / 1.0.0 · 2026 年 11 月 1 日

这份教程带你完成：准备环境、安装仓颉、安装本技能、首次提问、运行十一项教学模型及归档审计、复现一个完整实验、检查结果并安全升级。

项目：https://github.com/WEveboy/STL_Houjie-skill。发行标签：edition-v1.0.0。本教程的发行下载链接在相应 Release 实际发布后生效；若资产缺失，请使用仓库源码，并以发布记录为准。

目录

1 先认识两种不同的工具 2

2 从官方刷新包安装仓颉 2

3 把仓颉和 STL 成品接入 Codex 3

4 第一次提问：同时给任务、输入与边界 4

5 从项目根目录运行十一项教学模型及归档审计 5

6 完整实验：合法右移、故障对照与保存结果 7

7 再用真实 C++ 与项目检查区分模型和实现 8

8 怎样读这本书的历史边界 8

9 常见故障：从最小可确认的状态开始 9

10 升级、备份与退回 10

11 继续精读、重新蒸馏与维护 10

12 增量功能：不变量、重数、重排与实验审计 11

13 官方来源与随包阅读顺序 12

1 先认识两种不同的工具

仓颉 `cangjie-skill` 是作者端的蒸馏工具：读取长内容、提取候选、验证来源与执行条件，再编译技能。`stl-houjie` 是已经整理的 STL 成品技能：根据你的问题读能力卡、调用资源、解释源码与验证边界。**使用成品技能不以先装仓颉为硬前提**。教程从仓颉讲起，是为了使初次使用者也能理解来源与后续维护。

本项目研究侯捷书中早期 SGI STL 的设计，并提供原创的小规模教学模型和现代 C++17 契约实验。它不包含原书 PDF 或整章 OCR，不是完整 STL 实现，也不是能证明任意容器正确的静态分析器。你提出具体问题时，仍应给出目标库、编译器、标准版本和最小代码。

1.1 准备一个工作目录

下面 Windows PowerShell 示例使用公开示例目录 `C:/SkillLab`，你可换到自己的工作盘。先打开终端，确认 Python 可用；本项目模型只使用 Python 标准库。仓颉脚本另需 PyYAML，建议用独立虚拟环境隔离。

```
python --version
python -m pip --version
New-Item -ItemType Directory -Force C:/SkillLab
Set-Location C:/SkillLab
```

需要 Python 3.10 或以上。若 Windows 仅提供 `py`，可把本教程的 `python` 替换为 `py -3`，先确认它指向同一个安装。macOS/Linux 可使用 `python3`，其目录写法按系统调整。不要把不同 Python 的 pip 混在一起。

建议保留三类目录：下载包、解压项目、自己的实验输出。这样升级不会覆盖输入和结果。安装目录只保存成品资源，不应当作日常实验仓库使用。

2 从官方刷新包安装仓颉

本次作者工具基线是仓颉 2.5.0 刷新包。官方说明指出，2026-09-13 的刷新仍沿用版本号 2.5.0，自动生成的旧 tag 源码 ZIP 不包含该刷新。应从上游 README 的刷新包入口或 Release 资产下载，检查`BUILD_INFO.json`；不要只下载一个`SKILL.md`。

上游与版本页：

- https://github.com/kangarooking/cangjie-skill
- https://github.com/kangarooking/cangjie-skill/releases/tag/v2.5.0

刷新通用包名为`cangjie-skill-2.5.0.zip`，旁边有同名`.sha256` 校验文件。下载后先核对 SHA256，再解压，找到完整`cangjie-skill/` 目录；它应包含入口、`scripts/`、`methodology/` 和`schemas/` 等内容。不要误把外层解压目录当技能目录。

```
Get-FileHash ./cangjie-skill-2.5.0.zip -Algorithm SHA256
Get-Content ./cangjie-skill-2.5.0.zip.sha256
```

逐字符比对两个哈希值，不比较文件名字段。SHA256 说明下载字节与公布清单一致，不等于对发布者身份或内容安全作出独立认证。若不一致，停止使用该包，重新从确认的来源下载。

进入含`scripts/cangjie.py` 的解压目录，创建作者端环境并做诊断：

```
python -m venv .venv
./.venv/Scripts/python.exe -m pip install PyYAML
./.venv/Scripts/python.exe scripts/cangjie.py doctor
```

Linux/macOS 把虚拟环境解释器路径改为`.venv/bin/python`。遇到明确列出的额外依赖缺失，按 doctor 提示在同一环境安装；可选依赖警告与必需依赖失败要分开读。不要为了“消除警告”无目标地安装整套开发环境。

成功检查应能说明工具版本及必需依赖状态。把输出保存在自己的维护记录中。接下来的安装位置由平台决定；不要把解压位置、宿主技能发现目录和本项目 Release 根目录混为一谈。

3 把仓颉和 STL 成品接入 Codex

3.1 选择一个发现位置

按 2026-11-01 查阅的 Codex 官方技能文档，用户级目录可用`~/.agents/skills/`，项目级可用项目中的`.agents/skills/`。将完整仓颉目录放入其中一个位置，例如用户级最终路径为`~/.agents/skills/cangjie-skill/SKILL.md`。若已有同名入口，先检查版本及保存位置，备份后再有意更新。

项目级技能适合只在这个项目使用；用户级技能适合多个项目共享。不要同时放多份同名 `stl-houjie` 或仓颉入口来“增加成功率”，同名技能不会自动合并，可能造成选择混乱。官方文档为https://learn.chatgpt.com/docs/build-skills。

3.2 下载 STL 第一版并核验

进入项目 Release：https://github.com/WEveboy/STL_Houjie-skill/releases/tag/edition-v1.0.0。选择`stl-houjie-Codex-1.0.0.zip` 与`SHA256SUMS.txt`。如果你希望阅读完整源卡、精读稿或运行开发测试，也可下载源码快照。

```
Get-FileHash ./stl-houjie-Codex-1.0.0.zip -Algorithm SHA256
Get-Content ./SHA256SUMS.txt
```

确认对应文件行的哈希一致再解压。Codex 安装 ZIP 根目录直接包含`install_codex.py` 及完整`stl-houjie/` 目录，在这个解压根目录打开终端。源码项目则使用`scripts/install_codex.py` 与`skills/stl-houjie/` 布局，两者不要混用。安装器会检查成品清单，而不是只复制入口文字。

3.3 首次安装与项目安装

```
python install_codex.py --destination "$HOME/.agents/skills"
```

参数是 **skills 根目录**，不要传成.../skills/stl-houjie，否则会多嵌一层。项目级安装可先切换到自己的目标项目，再把那个项目的.agents/skills 绝对路径传入安装器。例如：

```
python install_codex.py --destination C:/SkillLab/MyProject/.agents/skills
```

安装成功时输出 JSON，status 为 installed_and_verified，包含版本、目标目录和可能的备份路径。如果已存在目标且没有--replace，安装器会拒绝覆盖；这是保护现有文件的预期行为。

3.4 确认宿主能找到技能

在 Codex 中开始一个适合使用该项目技能的新对话，检查技能列表，或直接发送下一节的显式提示词。官方说明技能变更可自动发现；若仍不出现，重启 Codex 再检查。不要在“技能未发现”时反复把同一目录复制到多个位置。

安装版资源位于实际安装目标的stl-houjie/resources/，能力卡在stl-houjie/references/capabilities/。先从安装器输出找到目标，再定位文件；不要假设电脑一定使用某个旧版私有目录。源码版本的相同资源在skills/stl-houjie/resources/。

后续实验统一使用源码布局。请另下载 Release 中的stl-houjie-source-1.0.0.zip，解压后进入stl-houjie-skills/，确认其中有scripts/ 和skills/。若从源码安装，命令改为 python scripts/install_codex.py，仍传同一个 skills 根目录。

4 第一次提问：同时给任务、输入与边界

首次调用推荐显式写技能名，使“是否找得到入口”和“任务是否做对”更容易分开。可以先发送：

> 使用 stl-houjie。按侯捷书中 SGI 模型解释 vector 为什么 size 与 capacity 不同；用 size=3、capacity=4、在 index=1 插入 2 项推演状态。区分原书实现策略与现代标准保证，给出可复算步骤。

如果宿主支持美元符号显式技能选择，也可写 $stl-houjie。关键是你看见宿主实际加载了正确入口和能力卡，而不是模型仅凭常识回答了同名主题。

一个有用的回答应明确初始有效范围、为何容量不足、新模型容量为 6 及新 size 为 5，并说明这是所列教学策略，不是所有现代 vector 的增长倍率。它不能仅以“通常翻倍”替代输入条件。若你问真实源码，请继续提供库名、版本、代码和观测结果。

常用提问模板：

- “分析这段 copy：标出源范围、目标范围、对象是否已存在、重叠方向，先判断前提再执行。”
- “按给定比较器解释 set/map 如何确定键等价；不要把 hash 碰撞当相等。”

- “画出 deque 的区号和区内偏移，B 以元素计，解释负位移；不要把 B 当字节。”
- “remove/unique 以后哪些位置是有效结果？输出新尾，别把 size 仍未变说成错误。”

若问题缺少必要条件，技能应提出针对性补充，或将例子明确标为合成演练；不应猜出一个看似合理的源码版本。若要求运行代码，宿主需具备相应本地工具权限；纯聊天结果不等于已执行。

5 从项目根目录运行十一项教学模型及归档审计

以下均为本项目原创、有预算限制的整数教学模型。安装包`resources/usage.md` 是输入合同的完整版本；此处给复制即可运行的起点。先找到发行项目根目录，再运行：

```
python skills/stl-houjie/resources/stl_lab.py demo
```

每次 CLI 调用输出一个 JSON。成功退出码 0；输入、前提或预算错误为退出码 2，且 `status` 为 `input_error`。`demo` 一次展示 8 项操作；它不是 8 次模型对话，也不要求联网。

5.1 run 方式：一个文件对应一次操作

把以下任意一段 JSON 保存为`input.json`，使用同一入口执行。用文本编辑器保存 UTF-8 即可；Windows UTF-8 BOM 也受支持。不要把 Markdown 代码围栏写入 JSON 文件。

```
python skills/stl-houjie/resources/stl_lab.py run input.json
```

5.2 1. pool-class：小请求分档

```
{"op":"pool-class","bytes":30}
```

30 字节按所列 SGI 参数上调到 32 字节，自由链表使用零下标 3。128 仍为小档，超过 128 走一级路径。模型不申请真实内存，也不模拟线程或任意对齐类型。

5.3 2. vector-growth：大小和容量状态

```
{"op":"vector-growth","size":3,"capacity":4,
 "insertions":[{"index":1,"count":2}]}
```

新 size 为 5；容量不足，教学重配置公式为旧 size 加 max(旧 size, 插入数)，得到 6。它不计算真实对象的复制/移动次数与异常保证，也不代表目标标准库的内部参数。

5.4 3. deque-step：分段位移

```
{"op":"deque-step","buffer_elements":8,"offset":1,"delta":-3}
```

从区内 1 向前退 3，到前一区的区内 6。参数 `buffer_elements` 以元素为单位；模型只给相对区位，不能证明该位置在某个真实 deque 里合法。

5.5 4. heap-trace：维护最大堆

```
{"op":"heap-trace","values":[9,7,8,2],"action":"pop"}
```

输入必须已经是有效的零下标最大堆。pop 把最大值 9 放到末尾，整体数组长度保持 4，有效堆范围缩短为 3。push 操作另给 `value`；不要把算法维护当成容器自动删除。

5.6 5. logical-end：保留有效前缀

```
{"op":"logical-end","values":[1,1,2,1],"action":"unique"}
```

有效前缀是 1、2、1，新尾为 3，原 size 为 4。unique 只压缩相邻重复。若 action 改为 remove，应另给 `value`；不需要的字段不能保留凑数。

5.7 6. bounds：有序范围的边界

```
{"op":"bounds","values":[1,2,2,4],"value":2}
```

非递减整数范围中，lower 为 1，upper 为 3，等价范围是 [1,3)。模型不接受无序输入或任意用户比较器，不会静默排序后假装验证了原范围。

5.8 7. hash-buckets：数学取模与重散列

```
{"op":"hash-buckets","values":[1,6,-1],"bucket_count":5,
 "new_bucket_count":7}
```

按非负数学余数分桶；重复值保留，改桶数后重新映射。它不仿真 `std::hash`，不承诺 SGI 或现代库的实际遍历顺序，负数处理约定要和真实 C++ 实现分开。

5.9 8. copy-plan：同一数组中的合法复制

```
{"op":"copy-plan","values":[1,2,3,4,5],"first":0,"last":3,
 "result":5,"direction":"backward"}
```

此处 result 是目标尾点，目标 [2,5)，结果为 1、2、1、2、3。forward 时 result 改为目标起点，不能只换 direction 而继续沿用同一个数字解释。非法重叠及目标对象不足会被拒绝，不执行未定义行为来观察输出。

输入预算为文件不超过 64KiB，数组/操作数最多 256 项，模型储存和桶数最多 4096，普通整数绝对值不超过 1000000。程序拒绝重复 JSON 键、NaN/Infinity、未知字段以及布尔值冒充整数。预算或合同不满足时，应先修正任务，而非通过删除校验来运行。

6 完整实验：合法右移、故障对照与保存结果

目标是把已有数组前三项右移到索引 [2,5)，保持仍需读取的源项不被提前覆盖。这个任务适合 `copy-plan` 及复制契约卡。它是自拟教学实验，不是原书某次运行的伪装复刻。

6.1 建立输入与执行

从项目根目录运行；PowerShell 的单引号 here-string 使 JSON 不受变量替换影响。

```
New-Item -ItemType Directory -Force experiments/copy-right
@'
{"op":"copy-plan","values":[1,2,3,4,5],"first":0,"last":3,
 "result":5,"direction":"backward"}
'@ | Set-Content -Encoding utf8 experiments/copy-right/input.json
python skills/stl-houjie/resources/stl_lab.py run experiments/copy-right/input.json >
    experiments/copy-right/output.json
```

读取`output.json`，核对最外层 status 为 ok，结果对象中 `values` 为 `[1,2,1,2,3]`，`destination` 为 `[2,5]`，`returned_index` 为 2。结果之外还应读 scope 和 contract，确认这是同一已有整数数组的有限模型。

6.2 手算核对，不只检查“程序没报错”

从尾向前赋值：先把索引 2 的 3 写到 4，再把索引 1 的 2 写到 3，最后把索引 0 的 1 写到 2。每次读取的源仍未被覆盖，最终前两项保留 1、2。返回索引 2 表示本次目标的起点；它不是删除后的容器新尾。

把方向改为 forward 并把 result 改成 2，就是同一向右覆盖的危险方向：result 落在源 [0,3) 内，模型应返回 input_error 与退出码 2。若保持 result=5 而改 forward，则它成了从索引 5 开始写三项，失败原因是目标越界。两个失败条件不同，修复应对准其输入语义，不要混称为“copy 都不能重叠”。

6.3 归档最小可复现记录

至少保留输入、完整输出、Python 版本、资源文件哈希及你核对过的断言：

```
python --version
Get-FileHash skills/stl-houjie/resources/stl_lab.py -Algorithm SHA256
Get-FileHash experiments/copy-right/input.json -Algorithm SHA256
Get-FileHash experiments/copy-right/output.json -Algorithm SHA256
```

哈希只帮助确认字节是否改变，不能代替上面的数学与合同核对。不要把凭据或私人绝对路径收进公开日志；与他人共享时，给项目版本、相对文件和必要环境就够了。

7 再用真实 C++ 与项目检查区分模型和实现

Python 模型有助于推演，却不证明某个 C++ 类型的对象生存期或迭代器操作合法。源码项目包含原创 C++17 契约实验，需要本机可用的 `g++`、`clang++` 或 `cl`。先让相应编译器在当前终端的 PATH 中可见；MSVC 通常要在配置好的开发者终端使用。

```
python scripts/run_cpp_checks.py
```

运行器记录实际编译器版本、编译与执行状态；默认结果保存在`build/cpp-checks/result.json`。若找不到源文件，可按`--help` 指定`--source` 和`--build-dir`。不建议双击 CPP 文件期待自动运行。

实验检查储存/构造分离、未初始化复制失败清理、traits 与迭代器能力、vector 重配置、list 接合、deque 访问、堆逻辑 pop、map/哈希键、复制方向、逻辑新尾、二分边界、含重复项的 nth_element、引用修改、反向 base 与插入输出等。它保持断言启用，不固定现代库的容量倍率或桶数，不解引用失效迭代器。

本版作者工作区已完成的检查规模为 Python 26 组、C++ 16 组、安装器 6 项。该数字只表示对应检查程序的范围，不是 18 项能力均经同一真实模型实测的证明。独立任务盲测、路由评测和来源审计以仓库随版报告为准；本教程不预填尚未得到的结果，也不声称比不使用 Skill 更准确。

从源码根目录可重新运行 Python 和安装器测试：

```
python -m unittest discover -s skills/stl-houjie/resources -p test_stl_lab.py -v
python -m unittest discover -s tests -v
```

若只有精简安装包而没有`tests/` 或`scripts/`，先下载源码项目，不要把“资源没有随这个包提供”误判为算法失败。运行成功后保存日志及版本，失败则先读第一处错误和当前环境，不用删除断言掩盖问题。

8 怎样读这本书的历史边界

先读`source/CHAPTER_READING.md` 建立八章关系，再按任务查`source/DIGEST.md` 和`skills/stl-houjie/references/capabilities/`。一次具体操作可按“储存哪里来—对象何时存在—迭代器能做什么—算法要求什么—策略与配接如何改变行为”追踪。

- SGI 的小块 128 字节阈值、8 字节分档、vector 增长公式、deque 区容量及哈希质数表都带实现限定，不能写成现代所有标准库必须采用的常量。
- 已分配储存不等于已有目标对象；reserve 后 size 仍为 0，不能据此向 begin 普通 copy 若干元素。
- remove/unique 给逻辑新尾而不自动缩小容器；pop_heap 不删除末项；unique 只处理相邻重复。
- deque 中的随机访问不保证整个序列是连续地址；反向迭代器的 base 不等于它正在解引用的位置。
- 原书 deque 部分单位与算式、nth_element 描述、for_each 简化分类有局部冲突，按来源审计分别处理；不要为“忠于原文”继续执行错误条件。
- 书中部分异常及线程路径被省略，不能从节录推导完整强保证或线程安全。完整红黑删除修复不在本版的原书插入讲解保证内。

若你研究 MyTinySTL 或另一现代库，先提供该仓库对应源码与提交；本技能能帮助建立分析框架，不能因为看过侯捷书就宣布另一个实现正确。现代接口及移除年代等事实需查当前官方标准或目标库资料，并与原书证据分开。

9 常见故障：从最小可确认的状态开始

找不到 python 或 pip

先重开终端并确认安装与 PATH；用 `python -m pip` 避免 pip 属于另一解释器。Python 模型无第三方依赖，只有仓颉作者端另需 PyYAML。

技能没有出现

检查是否整目录安装、入口是否多嵌一层、同名入口是否重复。Codex 检查所选发现位置；Harness 检查 profile 及注册日志。宿主未实际加载不能用模型口头声称替代。

stl_lab.py 路径不存在

先定位完整成品根目录。源码在`skills/stl-houjie/resources/`，安装版在安装器输出目标下，Harness 以注册的 resourceBase 为准。

JSON 返回 input_error

先读 message。检查字段名、整数与布尔值、范围、排序及堆序前提。UTF-8 BOM 受支持，代码围栏、单引号 JSON 和末尾多逗号不是有效输入。

得到和书中不同的 capacity 或 bucket 数

先核所用实现。教学模型、历史 SGI 输出和你本机标准库的参数不是同一个承诺；不要为了匹配书图修改测试目标。

C++ 实验找不到编译器

在配置好的开发者终端运行，检查编译器版本。返回 unavailable 不能写成 16 组已在你的机器通过；仅靠 Python 通过也不能补上这项执行。

插件加载后打不开资源

检查 TGZ 是否完整、注册时目录资源契约及宿主版本。不要只复制`index.mjs`；它依赖同包的技能及资源目录。

模型没有实际运行工具

请求它报告命令、退出码和产物位置；若宿主无本地工具，就自己按教程命令运行。模型回答的预期结果与实际输出应分开保存。

10 升级、备份与退回

下载新的成品及校验文件，先核验包的 SHA 和版本，再运行当前安装器。若本版已安装，有意升级使用：

```
python scripts/install_codex.py --destination "$HOME/.agents/skills" --replace
```

安装器先校验新成品，备份旧目录，再放入并核验新目录；输出的 backup 字段记录旧版位置。请记录这个路径及旧新版本，先运行 demo 与自己的核心输入，再决定何时清理旧版。不要递归删除不确定的计算目录。

回退应停止正在使用该目录的任务，保存当前目录，将已确认完整的备份恢复到原来的单一技能位置，再检查版本和 demo。不要让备份内的同名入口继续被宿主扫描而同时触发两版。用户自己的实验输入与输出应独立保存，不随技能升级替换。

11 继续精读、重新蒸馏与维护

若只想使用现有能力，到此即可。若想加入新的源码版本或补全未覆盖机制，先保留旧版，使用仓颉整理新增证据，不手改编译后入口冒充源数据更新。你可向作者端宿主发送：

> 用 cangjie-skill 继续维护本项目。先读 source 中的整书骨架、覆盖审计与流水线状态，再核对新材料。保留稳定能力 ID；只修改有来源或实际失败支持的卡片。分别记录来源核查、代表任务输出和边界测试，重新编译后再打包。

本项目的作者源与可编译 Bundle 同在`source/`，其中含 verified.yaml、cards、book 与 resources；不要把上游旧示例的侧车路径套到本发行目录。重新蒸馏需要合法可读的原始材料，公开仓库并不附带整本书。不能只凭模型记忆补出原书没有写的步骤或把现代补充归给作者。

维护循环为：更新关键任务与候选、检查来源充分性/可执行性/任务增益、回填 RIA 能力卡、补输出与边界评测、编译 single 成品、验证安装与两平台资源一致性、记录版本及发布。阅读稿负责解释全书，能力卡负责完成任务，模型脚本负责有限可复算部分，三者不互相替代。

发布前核对远端主分支并保留其他提交，只上传维护所需文件；原书、全文 OCR、页图、缓存、模型密钥与本地私人路径不进入公开仓库。Release 资产按版本发布并核对下载哈希。未完成的宿主测试或来源缺口要写明，不能用清单形式掩盖。

12 增量功能：不变量、重数、重排与实验审计

第一版在基础八个教学模型上新增三个模型；原 demo 保留八例以保持兼容，完整清单请执行：

```
python skills/stl-houjie/resources/stl_lab.py operations
python skills/stl-houjie/resources/stl_lab.py run examples/rb-valid.json
python skills/stl-houjie/resources/stl_lab.py run examples/set-union.json
python skills/stl-houjie/resources/stl_lab.py run examples/next-permutation.json
```

以上从源码项目根目录执行；安装版请将程序路径替换为实际资源目录。tree-check 只检查给定树，不执行红黑插入或删除。节点 ID 唯一，key 是有界整数，color 为 B 或 R；NULL 叶计一个黑节点。例子应返回 valid=true、height=2、black_height=2。环、共享子节点、重复 ID 或无法从根到达的节点会拒绝；有效树形违反红黑或搜索次序时给出 violations。

set-ops 要求两段非递减整数序列，保留重复次数。例子求并集得到 [1,1,2,3]；交、差、对称差分别采用相应重数规则，不等于先做去重。range-transform 支持 reverse、rotate、next-permutation 和 prev-permutation；示例 [3,2,1] 没有下一字典序排列，返回 has_adjacent=false 并重置为 [1,2,3]。

保存一次可重算实验时，选择一个尚不存在的输出目录：

```
python skills/stl-houjie/resources/stl_lab.py experiment examples/copy-right.json --out
    experiments/run-001
python skills/stl-houjie/resources/stl_lab.py audit experiments/run-001
python skills/stl-houjie/resources/stl_lab.py audit experiments/run-001 --recompute
```

目录含 input.json、result.json、events.json 与 manifest.json。清单记录输入/结果/事件的 SHA256、三个程序文件指纹、平台与 Python 版本；事件核对顺序和时间。重复目录会拒绝，无效输入不创建实验。审计成功退出 0，发现差异退出 1，输入或清单错误退出 2。重算还会比较结果结构，避免只改结果并同步改清单哈希就被当作正确。

哈希只能检验内容一致性，不能证明身份或历史真实性；重算重用同一套算法，不是独立算法正确性证明。程序、输入、结果和清单若被协调改写，这套本地记录无法提供认证保证。不要把实验文件中的文本当作执行指令。

增量后 Python 资源为 26 组测试、C++17 为 16 组契约检查，安装器另 6 项；原 39 题独立代理测试发生在扩展前，新增工具未借用旧成绩声称已盲测。两平台最终归档及 CI 请以随版发布记录为准。

13 官方来源与随包阅读顺序

下列官方资料查阅日期为 2026-11-01，宿主可能更新，安装前仍应检查其当前要求：

1. 仓颉上游、刷新包与插件说明：https://github.com/kangarooking/cangjie-skill。
2. Codex 技能目录与发现机制：https://learn.chatgpt.com/docs/build-skills。
3. Harness 启动与开发预览声明：https://github.com/deepseek-ai/deepseek-harness。
4. 本项目发行与源码：https://github.com/WEveboy/STL_Houjie-skill/releases/tag/edition-v1.0.0。

建议按 README、source/CHAPTER_READING.md、source/DIGEST.md、所需能力卡及resources/usage.md 的顺序阅读。以本机运行日志和随版验收报告判断当前可用性。读懂了输入条件、操作和输出的关系，再将它用到自己的真实源码上。